



Session 3:

Build Your First PoC (Part-2)

03 FINISH, DOCUMENT, STABILISE

Documentation-First Development

Prompt Claude Code to write the README as you build — not after. A PoC without a README is not handoff-ready.



Forces scope clarity

Writing what it does exposes ambiguity. If you cannot describe it clearly, it is not done.



Makes handoff instant

Anyone picking this up — engineering, a colleague, future you — can start without a briefing.



Confirms the build

Generating the README reveals gaps between what you intended and what Claude actually built.

THE README PROMPT

```
"Write a README.md for this project. Include:  
- What it does (1 paragraph)  
- Who uses it and when  
- How to install and run it  
- What a successful run looks like  
- What is out of scope"
```

Run this now — before you finish building. Edit what is wrong. Keep it current.

Error Handling Basics

The difference between a demo that breaks under questions and one that holds up.

Invalid or missing input

Without: Script crashes with an unreadable stack trace **With:** Clear message: what was expected, what was received

"Add input validation. If [INPUT] is missing or invalid, print a clear error and exit gracefully."

API or service unavailable

Without: Connection timeout with no explanation to the user **With:** Informative failure: what was tried, that it failed, what to check next

"Handle API connection errors. If the request fails, name the endpoint and suggest checking the API key."

Empty or unexpected response

Without: KeyError or AttributeError mid-execution **With:** Handled: logs the empty response, continues or exits cleanly — no crash

"Handle empty responses from [API]. If no results, print a message and exit — do not crash."

Your PoC is done when it answers the question it was built to answer.

✗ Not when it handles every edge case

✗ Not when it is polished or production-ready

✗ Not when you have simply run out of time

Go back to your success criterion. If it passes — you are done.

The Self-Review Prompt

Before you call it done — ask Claude to stress-test it. 30 seconds. Saves your demo.

THE PROMPT

```
"What are the top 3 things most likely to break or fail in a live demo of this PoC? Be specific."
```

Classify each risk Claude identifies:

Blocking

Breaks the demo entirely

→ Fix it now — before the commit

Deferrable

Edge case unlikely in a controlled demo

→ Note it in the README. Move on.

Known Limitation

Out of scope by design

→ Confirm it is in your 'out of scope' list.

Saving Your Work — Git Commit

Commit before Session 3. Claude generates the message. You review and push.

Generate the commit message

1

```
$ claude "Write a Git commit message for this session. Include what it does, what it does not do, and any known limitations."
```

Review it. Edit if vague.

Stage your files

2

```
$ git add .
```

Check git status first. Only stage files you intended to change.

Commit with the generated message

3

```
$ git commit -m "[CLAUDE-GENERATED MESSAGE]"
```

This PoC runs through Sessions 3 and 4. A clear message matters.

Live Build Part 2 — Complete, Document, Commit

HANDS-ON LAB

Finish the PoC. Make it explainable. Make it committable.

1

Complete the working core

If Part 1 left anything unfinished — finish it. One function, one clean output.

2

Run the self-review prompt

Fix anything blocking. Note what is deferrable in the README.

```
$ claude "Top 3 things most likely to fail in a live demo?"
```

3

Generate and review your README

Run the README prompt. Fix any inaccuracies before saving the file.

```
$ claude "Write a README.md for this project..."
```

4

Add basic error handling

Apply the three error handling prompts to your most likely failure points.

5

Commit to Git

Generate commit message · Stage files · Commit · You are done for Session 2.

```
$ git add . && git commit -m "[message]"
```

✓ **SUCCESS: PoC runs · Self-review done · README saved · Error handling added · Committed to Git**

60-Second Demo

SHARE-OUT

One or two volunteers. 60 seconds each. Show it running — not on slides.

COVER THREE THINGS IN 60 SECONDS

1

What question does your PoC answer?

One sentence. The problem it was built to solve.

2

Show it running.

Execute it live. Real output. Not a screenshot or a description.

3

What would you change with another 30 minutes?

The most important gap you are aware of.

Everyone else: note one thing you would borrow from what you just saw.

What You Now Have

SCOPED & BRIEFED

- ✓ PoC brief: one journey, one integration, one testable outcome
- ✓ Scope test passed
- ✓ CLAUDE.md created and verified

BUILT & RUNNING

- ✓ Working PoC — core logic runs
- ✓ Build loop applied: prompt → run → fix → repeat
- ✓ Self-review prompt run — blocking risks addressed

DOCUMENTED & COMMITTED

- ✓ README generated and reviewed
- ✓ Basic error handling added
- ✓ Committed to Git with a descriptive message

This PoC continues in Sessions 3 and 4. Keep it open — you will build on it.

What's Next — Session 3

Debug, Review & Improve Your Code · Same PoC. New layer.

AI-Assisted Debugging

01

Apply the 5-step debug framework. Read stack traces with Claude as co-pilot. Find and fix the failures before they reach a stakeholder.

AI-Assisted Code Review

02

Run a structured review prompt on your own PoC. Identify severity-ranked issues. Apply the fixes. Re-run the review to confirm.

Development Workflows

03

Commit clean work to Git with Claude-generated messages. Produce a one-page technical overview your engineering team can act on.

Bring: your committed PoC · your Git branch · your prompt log from today.

Q&A

